

Telegram Rumble BOT コード解説 + 言語ガイド（初心者向け）

作成日: 2026-05-25

1. この資料の目的

この資料は、あなたのBOTコードを初心者でも読めるように解説し、あわせて使っている言語の「読み方」「書き方の基本」をまとめたものです。

2. コード全体の見取り図

今回のBOTは大きく次の層でできています。

1. 入力層（Telegram）

- ユーザーが /battle, /buy, /settings などを入力

2. 処理層（Node.js + Telegraf）

- コマンドを解析して必要な処理を実行

3. データ層（SQLite + PostgreSQL）

- 設定や状態を保存

4. 外部連携層（Stripe Webhook）

- 決済完了イベントを受けてPremium化

3. ファイルごとの完全解説

3-1. src/index.js（最重要）

何をするファイル？

BOTのメイン処理です。いわば「司令塔」です。

この中でやっていること

- 環境変数の読み込み
- DB接続と初期化
- Telegramコマンド登録
- 管理者チェック
- バトル開始/終了
- Stripe決済セッション作成
- Webhook受信
- Premium有効化
- 多言語ヘルプ表示

初心者が読む順番

1. 定数定義（BOT_TOKEN など）
2. utility関数（isAdmin, ensureGroup など）
3. コマンド群（bot.command(...)）
4. 起動処理（bot.launch）

重要関数

- isAdmin(ctx)

- Telegram APIから管理者一覧を取り、実行ユーザーが管理者か判定
- ensureGroup(ctx)
 - グループがDBに存在するか確認し、なければ作成
- startBattle(...)
 - 募集メッセージ送信、参加ボタン表示、カウントダウン開始
- finishBattle(...)
 - 参加者一覧表示 → 演出処理 → 勝者決定
 - 勝者だけメンションするように調整済み
- startWebhookServer()
 - /stripe/webhook エンドポイントを起動
 - checkout.session.completed を受けてPremium化

3-2. src/db.js

何をする？

SQLiteテーブルを作成します。

主なテーブル

- groups: グループ基本情報（premium状態含む）
- settings: グループ設定
- licenses: ライセンス情報
- battles: バトル履歴
- players: 参加者
- schedules: 自動開催設定
- stripe_events: Webhook重複防止

なぜ必要？

BOTが再起動しても「設定」と「履歴」を残すため。

3-3. src/services/i18n.js

何をする？

言語判定と翻訳文取得。

ポイント

- Telegramの language_code を見て ja/en/zh 判定
- t(lang, key) で翻訳テキストを返す
- effect_1, effect_2... を自動取得

3-4. src/services/utls.js

何をする？

時間やスケジュール文字列をパースする補助。

例:

- /schedule 21:00
- /schedule weekly mon 21:00
- /schedule once 2026-05-30 21:00

3-5. src/locales/ja.json, en.json, zh.json

何をする？

表示文言を言語別で管理。

書き方

- key: 値 形式
- keyは3言語で一致させる

例:

- battle_started
- join_button
- winner

4. コマンド設計の考え方

グループ向け

- /battle, /settings, /schedule など
- 管理者限定にして荒らしを防止

DM向け

- /helpdm, /buy -100xxxx
- 個人情報をグループに出さない

セキュリティ

- 権限チェック
- 重複イベント防止
- Webhook署名検証

5. 使用言語の読み方・書き方

5-1. JavaScript（ジャバスクリプト）

何の言語？

Webでもサーバーでも使える汎用言語。

基本文法（書き方）

1. 変数

- const: 再代入しない値
- let: 再代入する値

2. 関数

- function foo() {}

- const foo = () => {}

3. 条件分岐

- if (...) { ... } else { ... }

4. 配列操作

- map, filter, reduce

読むコツ

- まず「関数名」を見る
- 次に「引数」と「戻り値」を見る
- 最後に中身を追う

5-2. SQL (エスキューエル)

何の言語？

データベースを操作する言語。

基本文

- CREATE TABLE: テーブル作成
- SELECT: 取得
- INSERT: 追加
- UPDATE: 更新
- DELETE: 削除

今回の使い方

- groups/settings の保存
- プレイヤー一覧取得
- Premium状態の更新

5-3. JSON (ジェイソン)

何の形式？

キーと値でデータを表現する形式。

書き方

- 文字列はダブルクォート
- 末尾カンマを付けない

例:

```
{  
  "winner": "Winner: {user}"  
}
```

5-4. Markdown (マークダウン)

何の記法？

読みやすい文章を簡単に書くための記法。

基本
- # 見出し
- - 箇条書き
- `` コードブロック

6. 環境変数（.env / Railway Variables）の意味

代表例
- BOT_TOKEN: Telegram BOTの本体鍵
- STRIPE_SECRET_KEY: Stripe APIの秘密鍵
- STRIPE_WEBHOOK_SECRET: Webhook検証鍵
- STRIPE_PRICE_ID: 商品価格ID
- DATABASE_URL: PostgreSQL接続先
- BOT_OWNER_ID: オーナーID（復旧コマンド用）

注意
- トークン類は公開しない
- 変更後は再デプロイ

7. よくあるエラーと読み方

7-1. 409 Conflict
意味: BOTトークンが二重起動している。
対処: トークン再発行、1環境運用に統一。

7-2. Invalid License
意味: コードが未登録または使用済み。
対処: ライセンス発行元とコード形式を確認。

7-3. Premium OFF
意味: 保存先の永続化問題か対象グループ違い。
対処: groupid確認、DATABASE_URL確認、必要なら /premiumon。

8. 初心者向けの読み進め手順

1. まず /help で機能全体を把握
2. 次に src/index.js のコマンド部だけ読む
3. その後 db.js で保存先の構造を確認
4. 最後に i18n.js と locales を読む

この順番だと理解しやすいです。

9. 今後の学習ロードマップ

1. JavaScript基礎（関数、配列、非同期）

2. SQL基礎 (SELECT/INSERT/UPDATE)
3. API/Webhook基礎
4. エラーログの読み方
5. テストの書き方

10. まとめ

このBOTは、初心者でも次のポイントを押さえれば理解できます。

1. コマンドを受ける (Telegraf)
2. 処理する (JavaScript)
3. 保存する (SQLite/PostgreSQL)
4. 決済連携する (Stripe + Webhook)

この4層の理解ができれば、機能追加や運用判断がかなり楽になります。